

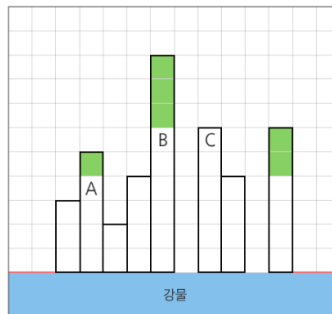
Doosan Robotics

Study

2025.04.23

이재호

• 문제



A와 B로 표시된 세대의 경우는 왼쪽 조망은 2칸 이상 확보가 되었지만 오른쪽 조망은 한 칸 밖에 확보가 되지 않으므로 조망권을 확보하지 못하였다.

C의 경우는 반대로 오른쪽 조망은 2칸이 확보가 되었지만 왼쪽 조망이 한 칸 밖에 확보되지 않았다.

[제약 사항]

가로 길이는 항상 1000이하로 주어진다.

맨 왼쪽 두 칸과 맨 오른쪽 두 칸에는 건물이 지어지지 않는다. (예시에서 빨간색 땅 부분)

각 빌딩의 높이는 최대 255이다.

[입력]

총 10개의 테스트케이스가 주어진다.

각 테스트케이스의 첫 번째 줄에는 건물의 개수 N 이 주어진다. ($4 \leq N \leq 1000$)

그 다음 줄에는 N 개의 건물의 높이가 주어진다. ($0 \leq$ 각 건물의 높이 ≤ 255)

맨 왼쪽 두 칸과 맨 오른쪽 두 칸에 있는 건물은 항상 높이가 0이다. (예시에서 빨간색 땅 부분)

[출력]

#부호와 함께 테스트케이스의 번호를 출력하고, 공백 문자 후 조망권이 확보된 세대의 수를 출력한다.

입력

```
10
0 0 254 185 76 227 84 175 0 0
10
0 0 251 199 176 27 184 75 0 0
11
0 0 118 90 243 178 99 100 200 0 0
```

출력

```
#1 111
#2 60
#3 165
...
```

• 해설

```
import sys
sys.stdin = open('sample_input.txt', 'r')

T = 10
# 여러개의 테스트 케이스가 주어지므로, 각각을 처리합니다.
for test_case in range(1, T + 1):
    N = int(input()) # num_building 4<= N <= 1000
    height = list(map(int, input().split()))
    sum = 0
    for i in range(N):
        if i == 0 or i == 1 or i == N-2 or i == N-1:
            continue
        else:
            near_max = max(height[i-2], height[i-1], height[i+1], height[i+2])
            sum += height[i] - min(near_max, height[i])

    print('#', test_case, ' ', sum, sep=' ')
```

Next Problem

https://swexpertacademy.com/main/code/problem/problemDetail.do?problemLevel=4&contestProbId=AV15KhN6AN0CFAYD&categoryId=AV15KhN6AN0CFAYD&categoryType=CODE&problemTitle=&orderBy=FIRST_REG_DATETIME&selectCodeLang=ALL&select-1=4&pageSize=10&pageIndex=2

https://swexpertacademy.com/main/code/problem/problemDetail.do?contestProbId=AV134DPqAA8CFAYh&categoryId=AV134DPqAA8CFAYh&categoryType=CODE&problemTitle=&orderBy=FIRST_REG_DATETIME&selectCodeLang=ALL&select-1=4&pageSize=10&pageIndex=1&&&&&&&

주요 용어 정리

- Position – 위치
- Orientation – 방위
- Frame – 계
- Kinematics: 운동학 (힘을 고려하지 않고 위치, 속도, 가속도의 관계를 다룸)
- Link: 관절과 관절을 연결하는 부
- Joint: 관절부
- Degree of Freedom: 물체의 상태를 최소한으로 표시하는 독립된 변수
- End-effector: 말단효과장치 (손과 같은 역할)
- Forward Kinematics: End-effector의 위치와 방위를 계산하는 운동학
- **Inverse Kinematics: End-effector로부터 관절의 위치, 각도를 구하는 것**
- Work space: Kinematics의 solution space
- Jacobian: 속도가 있을 때 다루는 행렬값.
 - 관절 공간에서의 속도를 직교좌표 공간속도로 매핑하는 것.
 - Joint Torque 계산에 필요
- Singularity: mapping의 inverse를 구할 수 없는 경우
- Dynamics: 운동을 일으키는 데 필요한 힘에 대한 학문
- Trajectory generation
- Cartesian coordinate system
- Polar coordinate system

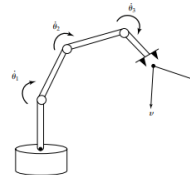


FIGURE 1.8: The geometrical relationship between joint rates and velocity of the end-effector can be described in a matrix called the Jacobian.

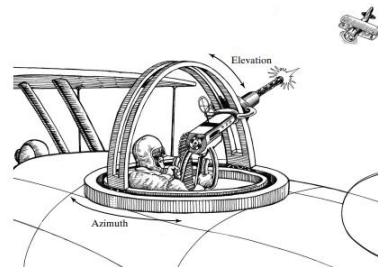


FIGURE 1.9: A World War I biplane with a pilot and a rear gunner. The rear-gunner mechanism is subject to the problem of singular positions.

Spatial Transformations

- Description of a position
 - Position vector

$${}^A P = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}.$$

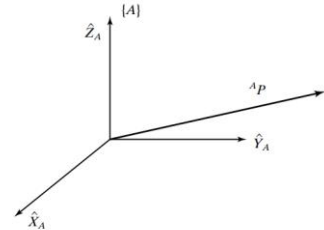


FIGURE 2.1: Vector relative to frame (example).

- Description of an orientation
 - Rotation matrix {B} relative to {A}
 - Direction cosines
 - Orthonormal

$${}^A R_B = \begin{bmatrix} {}^A \hat{X}_B & {}^A \hat{Y}_B & {}^A \hat{Z}_B \end{bmatrix} = \begin{bmatrix} \hat{X}_B \cdot \hat{X}_A & \hat{X}_B \cdot \hat{Y}_A & \hat{X}_B \cdot \hat{Z}_A \\ \hat{Y}_B \cdot \hat{X}_A & \hat{Y}_B \cdot \hat{Y}_A & \hat{Y}_B \cdot \hat{Z}_A \\ \hat{Z}_B \cdot \hat{X}_A & \hat{Z}_B \cdot \hat{Y}_A & \hat{Z}_B \cdot \hat{Z}_A \end{bmatrix}.$$

$${}^A R_B = \begin{bmatrix} {}^A \hat{X}_B & {}^A \hat{Y}_B & {}^A \hat{Z}_B \end{bmatrix} = \begin{bmatrix} {}^B \hat{X}_A^T \\ {}^B \hat{Y}_A^T \\ {}^B \hat{Z}_A^T \end{bmatrix}.$$

$${}^A R_B = {}^B R_A^{-1} = {}^B R_A^T$$

Spatial Transformations

- Description of a frame
 - Position (Rotation Matrix) and orientation

$$\{B\} = \{{}^A_B R, {}^A P_{BORG}\}.$$

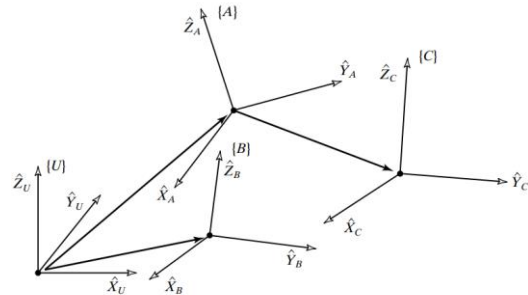


FIGURE 2.3: Example of several frames.

- Mapping
 - Changing from a frame to another frame
 - Translation (translated frames)

$${}^A P = {}^B P + {}^A P_{BORG}$$

- Rotation (rotated frames)

$${}^A p_x = {}^B \hat{x}_A \cdot {}^B P,$$

$${}^A p_y = {}^B \hat{y}_A \cdot {}^B P,$$

$${}^A p_z = {}^B \hat{z}_A \cdot {}^B P.$$

$${}^A P = {}^A_B R {}^B P$$

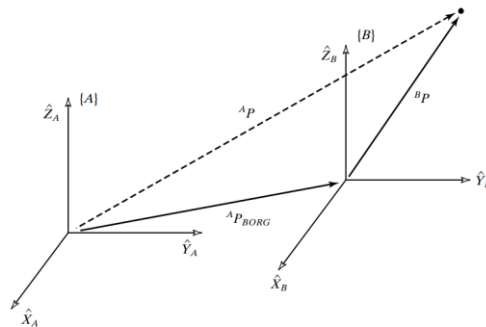


FIGURE 2.4: Translational mapping.

Spatial Transformations

- Mapping
 - General mapping

$${}^A P = {}^A R {}^B P + {}^A P_{BORG}$$

$${}^A P = {}^A T {}^B P$$

$$\begin{bmatrix} {}^A P \\ 1 \end{bmatrix} = \begin{bmatrix} {}^A R & | & {}^A P_{BORG} \\ \hline 0 & 0 & 0 & | & 1 \end{bmatrix} \begin{bmatrix} {}^B P \\ 1 \end{bmatrix}$$

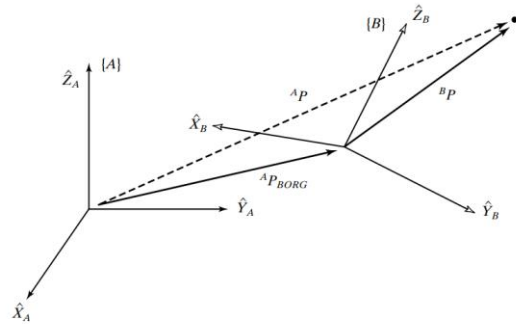


FIGURE 2.7: General transform of a vector.